

1. 目标

`hikari_loclite` 是一个独立 ROS2 定位包，不在现有 `lightning` full 框架里继续裁剪。它只复用现有项目的工程经验和少量可抽取算法代码，目标是面向嵌入式设备提供一条轻量、可控、可测的定位链路。

核心链路：

```
常态 GOOD:
Livox/PointCloud2 + IMU
-> Fast-LIO fixed-map tracking
-> Lite pose gate / smoother
-> TF / odom / loc_state

初始化 / LOST:
/initialpose 或 SC/NDT 产生初值
-> NDT validation
-> Fast-LIO ResetToMapPose()
-> fixed-map tracking
```

明确不搬运 full 框架：

- 不搬 PGO。
- 不搬动态地图。
- 不搬 Pangolin UI。
- 不搬 KISS 作为默认能力。
- 不搬 LidarLoc 全量状态机。
- 不保留 SC/KISS 常驻后台线程。

2. 包定位

建议包名：

```
hikari_loclite
```

建议放置路径：

```
/home/aecriclin/3d_slam_ws/src/hikari_loclite
```

不要建在 `lightning-lm` 内部。原因：

- 避免继承 `lightning` 现有 CMake 里 full 框架、UI、KISS、PGO、miao/g2o 的复杂依赖。
- 避免 `lightning.libs` 一次性链接太多目标。
- 便于给嵌入式部署做独立 deb、systemd、setcap 和参数模板。

3. 推荐目录结构

```
hikari_loclite/
  CMakeLists.txt
  package.xml
  config/
```

```
  loclite_livox.yaml
launch/
  loclite.launch.py
include/hikari_loclite/
  common/
    eigen_types.hpp
    point_types.hpp
    nav_state.hpp
  lio/
    fast_lio_fixed_map.hpp
    fixed_map_loader.hpp
    eskf.hpp
    ivox_adapter.hpp
  ndt/
    ndt_corrector.hpp
  reloc/
    reloc_manager.hpp
    scan_context_relocator.hpp
  system/
    loclite_node.hpp
    loclite_state_machine.hpp
    lite_pose_smoother.hpp
    ros_conversions.hpp
src/
  app/
    run_loclite_online.cpp
  lio/
    fast_lio_fixed_map.cpp
    fixed_map_loader.cpp
    eskf.cpp
    ivox_adapter.cpp
  ndt/
    ndt_corrector.cpp
  reloc/
    reloc_manager.cpp
    scan_context_relocator.cpp
  system/
    loclite_node.cpp
    loclite_state_machine.cpp
    lite_pose_smoother.cpp
    ros_conversions.cpp
thirdparty/
  nanoflann/
  ivox3d/
```

第一版可以更小:

```
hikari_loclite/
include/hikari_loclite/
  loclite_node.hpp
  fast_lio_fixed_map.hpp
  ndt_corrector.hpp
  lite_pose_smoother.hpp
src/
  run_loclite_online.cpp
  loclite_node.cpp
  fast_lio_fixed_map.cpp
```

```
ndt_corrector.cpp
lite_pose_smoother.cpp
```

4. 依赖策略

4.1 推荐依赖

第一版只引入：

- `rclcpp`
- `sensor_msgs`
- `nav_msgs`
- `geometry_msgs`
- `std_msgs`
- `std_srvs`
- `tf2`
- `tf2_ros`
- `tf2_geometry_msgs`
- `pcl_conversions`
- `pcl_ros`
- `livox_ros_driver2`
- `yaml-cpp`
- `PCL`
- `Eigen3`
- `OpenMP`

不建议第一版引入：

- `Pangolin`
- `OpenCV highgui`
- `KISS-Matcher`
- `miao optimizer`
- `g2o`
- `rosbag2_cpp`
- `visualization_msgs`

—— 4.2 代码复用方式

推荐从 `lightning-lm` 抽取最小代码，而不是链接 `lightning.libs`：

可抽取：

```
common/eigen_types.h
common/nav_state.h / nav_state.cc
common/point_def.h
core/lio/aa_faster_lio/eskf.*
core/lio/aa_faster_lio/imu_processing.hpp
core/lio/pointcloud_preprocess.*
core/lio/scan_context.*           # 只在需要 SC 时抽
core/localization/lidar_loc/pclomp/* # 只抽 NDT OMP, 别抽 LidarLoc full
thirdparty/nanoflann
core/ivox3d or thirdparty point_lio ivox
```

不建议抽取：

```
core/system/loc_system.*
core/localization/localization.*
```

```
core/localization/pose_graph/*
core/maps/tiled_map.*
ui/*
kiss_matcher_wrapper.*
```

这样 `hikari_loc_lite` 的编译和运行边界会清楚很多。

5. package.xml 片段

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>hikari_loc_lite</name>
  <version>0.1.0</version>
  <description>Lightweight fixed-map LiDAR localization for embedded deployment.
</description>
  <maintainer email="dev@example.com">hikari</maintainer>
  <license>Proprietary</license>

  <buildtool_depend>ament_cmake</buildtool_depend>

  <depend>roscpp</depend>
  <depend>std_msgs</depend>
  <depend>sensor_msgs</depend>
  <depend>geometry_msgs</depend>
  <depend>nav_msgs</depend>
  <depend>std_srvs</depend>
  <depend>tf2</depend>
  <depend>tf2_ros</depend>
  <depend>tf2_geometry_msgs</depend>
  <depend>pcl_conversions</depend>
  <depend>pcl_ros</depend>
  <depend>livox_ros_driver2</depend>

  <exec_depend>libcap2-bin</exec_depend>

  <test_depend>ament_lint_auto</test_depend>
  <test_depend>ament_lint_common</test_depend>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

6. CMakeLists.txt 片段

```
cmake_minimum_required(VERSION 3.16)
project(hikari_loc_lite)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_BUILD_TYPE Release)

add_compile_definitions(PCL_NO_PRECOMPILE)
```

```
find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)
find_package(sensor_msgs REQUIRED)
find_package(geometry_msgs REQUIRED)
find_package(nav_msgs REQUIRED)
find_package(std_srvs REQUIRED)
find_package(tf2 REQUIRED)
find_package(tf2_ros REQUIRED)
find_package(tf2_geometry_msgs REQUIRED)
find_package(pcl_conversions REQUIRED)
find_package(pcl_ros REQUIRED)
find_package(livox_ros_driver2 REQUIRED)
find_package(Eigen3 REQUIRED)
find_package(PCL REQUIRED COMPONENTS common io filters registration kdtree)
find_package(yaml-cpp REQUIRED)
find_package(OpenMP)

include_directories(
  include
  ${EIGEN3_INCLUDE_DIRS}
  ${PCL_INCLUDE_DIRS}
  thirdparty/nanoflann
)

add_library(${PROJECT_NAME}_core SHARED
  src/system/loclite_node.cpp
  src/system/loclite_state_machine.cpp
  src/system/lite_pose_smoother.cpp
  src/system/ros_conversions.cpp
  src/lio/fast_lio_fixed_map.cpp
  src/lio/fixed_map_loader.cpp
  src/lio/ekf.cpp
  src/ndt/ndt_corrector.cpp
  src/reloc/reloc_manager.cpp
  src/reloc/scan_context_relocator.cpp
)

ament_target_dependencies(${PROJECT_NAME}_core
  rclcpp
  std_msgs
  sensor_msgs
  geometry_msgs
  nav_msgs
  std_srvs
  tf2
  tf2_ros
  tf2_geometry_msgs
  pcl_conversions
  pcl_ros
  livox_ros_driver2
)

target_link_libraries(${PROJECT_NAME}_core
  yaml-cpp
  ${PCL_LIBRARIES}
)

if(OpenMP_CXX_FOUND)
```

```

    target_link_libraries(${PROJECT_NAME}_core OpenMP::OpenMP_CXX)
endif()

add_executable(run_loclite_online
  src/app/run_loclite_online.cpp
)

target_link_libraries(run_loclite_online
  ${PROJECT_NAME}_core
)

ament_target_dependencies(run_loclite_online rclcpp)

install(TARGETS
  ${PROJECT_NAME}_core
  run_loclite_online
  DESTINATION lib/${PROJECT_NAME}
)

install(DIRECTORY config launch
  DESTINATION share/${PROJECT_NAME}
)

ament_package()

```

7. 配置文件片段

config/loclite_livox.yaml:

```

common:
  livox_lidar_topic: "/livox/lidar"
  pointcloud_topic: "/cloud"
  imu_topic: "/livox/imu"
  lidar_frame_id: "livox_frame"
  map_frame_id: "map"

runtime:
  use_livox_custom_msg: true
  publish_tf: true
  publish_odom: true
  publish_path: false
  publish_debug_markers: false

fixed_map:
  enabled: true
  global_pcd: "/home/ubuntu/maps/site/global.pcd"
  voxel_leaf: 0.2
  crop_radius_m: 30.0
  rebuild_on_reloc: true
  max_points: 800000

fast_lio:
  filter_size_scan: 0.2
  ivox_grid_resolution: 0.2
  ivox_nearby_type: 18
  max_iterations: 4
  min_effective_points: 20
  blind: 0.3

```

```

lidar_type: 1
point_filter_num: 2
extrinsic_T: [-0.011, -0.02329, 0.04412]
extrinsic_R: [1,0,0, 0,1,0, 0,0,1]

ndt:
  enabled: true
  threads: 1
  resolution: 1.0
  max_iterations: 10
  correction_rate_hz: 1.0
  lost_rate_hz: 3.0
  min_confidence: 1.0
  max_delta_trans_m: 1.0
  max_delta_rot_deg: 10.0
  apply_gain_good: 0.1
  apply_gain_degraded: 0.3

reloc:
  auto_on_init: true
  auto_on_lost: true
  disable_after_good: true
  max_runtime_sec: 10.0
  sc_enabled: true
  sc_database: "/home/ubuntu/maps/site/sc_database.bin"
  poses_txt: "/home/ubuntu/maps/site/poses.txt"
  sc_top_k: 1
  sc_cooldown_sec: 5.0
  kiss_enabled: false
  fp_enabled: false

smoother:
  enabled: true
  max_correction_trans_m: 0.3
  max_correction_rot_deg: 5.0
  max_output_jump_trans_m: 0.5
  max_output_jump_rot_deg: 15.0

```

8. Launch 文件片段

`launch/loclite.launch.py`:

```

from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():
    config = LaunchConfiguration("config")

    return LaunchDescription([
        DeclareLaunchArgument(
            "config",
            default_value="/home/ubuntu/hikari_loclite/config/loclite_livox.yaml",
        ),
        Node(
            package="hikari_loclite",

```

```

        executable="run_loc-lite_online",
        name="hikari_loc-lite",
        output="screen",
        parameters=[{"config_path": config}],
    ),
]

```

9. 入口代码片段

src/app/run_loc-lite_online.cpp:

```

#include <memory>

#include <rclcpp/rclcpp.hpp>

#include "hikari_loc-lite/system/loc-lite_node.hpp"

int main(int argc, char** argv) {
    rclcpp::init(argc, argv);

    auto node = std::make_shared<hikari::loc-lite::LocLiteNode>();

    if (!node->Init()) {
        RCLCPP_ERROR(node->get_logger(), "failed to init hikari_loc-lite");
        rclcpp::shutdown();
        return 1;
    }

    rclcpp::spin(node);
    node->Shutdown();
    rclcpp::shutdown();
    return 0;
}

```

10. Node 骨架

include/hikari_loc-lite/system/loc-lite_node.hpp:

```

#pragma once

#include <memory>
#include <mutex>
#include <string>

#include <geometry_msgs/msg/pose_with_covariance_stamped.hpp>
#include <livox_ros_driver2/msg/custom_msg.hpp>
#include <nav_msgs/msg/odometry.hpp>
#include <rclcpp/rclcpp.hpp>
#include <sensor_msgs/msg/imu.hpp>
#include <sensor_msgs/msg/point_cloud2.hpp>
#include <std_msgs/msg/int32.hpp>
#include <tf2_ros/transform_broadcaster.h>

#include "hikari_loc-lite/livox/fast_livox_fixed_map.hpp"
#include "hikari_loc-lite/ndt/ndt_corrector.hpp"

```



```

#include "hikari_loclite/reloc/reloc_manager.hpp"
#include "hikari_loclite/system/lite_pose_smoother.hpp"
#include "hikari_loclite/system/loclite_state_machine.hpp"

namespace hikari::loclite {

class LocLiteNode : public rclcpp::Node {
public:
    LocLiteNode();
    bool Init();
    void Shutdown();

private:
    void OnImu(sensor_msgs::msg::Imu::SharedPtr msg);
    void OnLivox(livox_ros_driver2::msg::CustomMsg::SharedPtr msg);
    void OnCloud(sensor_msgs::msg::PointCloud2::SharedPtr msg);
    void OnInitialPose(geometry_msgs::msg::PoseWithCovarianceStamped::SharedPtr msg);

    void ProcessFrame();
    void PublishPose(const NavState& state);
    bool TryNdtCorrection(const NavState& fast_lio_state);
    void EnterLost(const char* reason);

    std::string config_path_;
    std::mutex mutex_;

    FastLioFixedMap::Ptr lio_;
    NdtCorrector::Ptr ndt_;
    RelocManager::Ptr reloc_;
    LitePoseSmoother smoother_;
    LocLiteStateMachine state_machine_;

    rclcpp::Subscription<sensor_msgs::msg::Imu>::SharedPtr imu_sub_;
    rclcpp::Subscription<livox_ros_driver2::msg::CustomMsg>::SharedPtr livox_sub_;
    rclcpp::Subscription<sensor_msgs::msg::PointCloud2>::SharedPtr cloud_sub_;
    rclcpp::Subscription<geometry_msgs::msg::PoseWithCovarianceStamped>::SharedPtr
initial_pose_sub_;

    rclcpp::Publisher<nav_msgs::msg::Odometry>::SharedPtr odom_pub_;
    rclcpp::Publisher<std_msgs::msg::Int32>::SharedPtr state_pub_;
    std::shared_ptr<tf2_ros::TransformBroadcaster> tf_pub_;
};

} // namespace hikari::loclite

```

src/system/loclite_node.cpp 核心流程:

```

bool LocLiteNode::Init() {
    this->declare_parameter<std::string>("config_path", "");
    this->get_parameter("config_path", config_path_);

    lio_ = std::make_shared<FastLioFixedMap>();
    ndt_ = std::make_shared<NdtCorrector>();
    reloc_ = std::make_shared<RelocManager>();

    if (!lio_->Init(config_path_)) return false;
    if (!lio_->LoadFixedMapFromConfig(config_path_)) return false;
    if (!ndt_->Init(config_path_)) return false;

```

```

if (!reloc_->Init(config_path_)) return false;

reloc_->Disarm("startup");
if (reloc_->AutoOnInit()) {
    reloc_->Arm("init");
}

imu_sub_ = create_subscription<sensor_msgs::msg::Imu>(
    "/livox/imu", rclcpp::QoS(100),
    [this](sensor_msgs::msg::Imu::SharedPtr msg) { OnImu(std::move(msg)); });

livox_sub_ = create_subscription<livox_ros_driver2::msg::CustomMsg>(
    "/livox/lidar", rclcpp::QoS(5).best_effort(),
    [this](livox_ros_driver2::msg::CustomMsg::SharedPtr msg) {
OnLivox(std::move(msg)); });

initial_pose_sub_ =
create_subscription<geometry_msgs::msg::PoseWithCovarianceStamped>(
    "/initialpose", rclcpp::QoS(5),
    [this](geometry_msgs::msg::PoseWithCovarianceStamped::SharedPtr msg) {
    OnInitialPose(std::move(msg));
});

odom_pub_ = create_publisher<nav_msgs::msg::Odometry>("hikari_loc_lite/odom", 10);
state_pub_ = create_publisher<std_msgs::msg::Int32>("hikari_loc_lite/state", 10);
tf_pub_ = std::make_shared<tf2_ros::TransformBroadcaster>(*this);
return true;
}

void LocLiteNode::OnImu(sensor_msgs::msg::Imu::SharedPtr msg) {
    std::lock_guard<std::mutex> lk(mutex_);
    lio_->AddImu(msg);
}

void LocLiteNode::OnLivox(livox_ros_driver2::msg::CustomMsg::SharedPtr msg) {
    {
        std::lock_guard<std::mutex> lk(mutex_);
        lio_->AddLivox(msg);
    }
    ProcessFrame();
}

void LocLiteNode::ProcessFrame() {
    NavState state;
    {
        std::lock_guard<std::mutex> lk(mutex_);
        if (!lio_->RunOnce(&state)) return;
    }

    const auto loc_state = state_machine_.State();

    if (loc_state == LiteLocState::Good) {
        TryNdtCorrection(state);
        reloc_->Disarm("good");
    } else if (loc_state == LiteLocState::Lost) {
        reloc_->Arm("lost");
        auto candidate = reloc_->TryRelocalize(lio_->LatestDeskewedCloud());
        if (candidate.valid && ndt_->Validate(candidate.pose, lio_-
>LatestDeskewedCloud()).valid) {
            std::lock_guard<std::mutex> lk(mutex_);

```

```

        lio_->ResetToMapPose(candidate.pose);
        state_machine_.SetGood("reloc_success");
        reloc_->Disarm("reloc_success");
    }
}

PublishPose(state);
}

```

11. 状态机片段

```

#pragma once

#include <string>

namespace hikari::loclite {

enum class LiteLocState {
    Uninitialized = 0,
    Initializing = 1,
    Good = 2,
    Degraded = 3,
    Lost = 4,
    WaitForInitialPose = 5,
};

class LocLiteStateMachine {
public:
    LiteLocState State() const { return state_; }

    void SetInitializing(const char* reason) {
        state_ = LiteLocState::Initializing;
        reason_ = reason ? reason : "";
    }

    void SetGood(const char* reason) {
        state_ = LiteLocState::Good;
        bad_count_ = 0;
        reason_ = reason ? reason : "";
    }

    void ObserveTrackingQuality(bool good) {
        if (state_ != LiteLocState::Good && state_ != LiteLocState::Degraded) return;
        if (good) {
            if (state_ == LiteLocState::Degraded && ++good_count_ >= recover_good_frames_)
            {
                SetGood("quality_recovered");
            }
            bad_count_ = 0;
            return;
        }

        good_count_ = 0;
        ++bad_count_;
        if (bad_count_ >= lost_bad_frames_) {
            state_ = LiteLocState::Lost;
            reason_ = "quality_lost";
        } else if (bad_count_ >= degraded_bad_frames_) {

```

```

        state_ = LiteLocState::Degraded;
        reason_ = "quality_degraded";
    }
}

private:
    LiteLocState state_ = LiteLocState::Uninitialized;
    std::string reason_;
    int bad_count_ = 0;
    int good_count_ = 0;
    int degraded_bad_frames_ = 3;
    int lost_bad_frames_ = 10;
    int recover_good_frames_ = 5;
};

} // namespace hikari::loclite

```

12. Fast-LIO fixed-map 接口片段

```

#pragma once

#include <memory>
#include <string>

#include <livox_ros_driver2/msg/custom_msg.hpp>
#include <sensor_msgs/msg/imu.hpp>
#include <sensor_msgs/msg/point_cloud2.hpp>

#include "hikari_loclite/common/nav_state.hpp"
#include "hikari_loclite/common/point_types.hpp"

namespace hikari::loclite {

class FastLioFixedMap {
public:
    using Ptr = std::shared_ptr<FastLioFixedMap>;

    bool Init(const std::string& yaml_path);
    bool LoadFixedMapFromConfig(const std::string& yaml_path);
    bool LoadFixedMap(const std::string& pcd_path, double voxel_leaf);
    bool RebuildLocalMapAround(const SE3& T_map_lidar);
    bool ResetToMapPose(const SE3& T_map_lidar);

    void AddImu(const sensor_msgs::msg::Imu::SharedPtr& imu);
    void AddCloud(const sensor_msgs::msg::PointCloud2::SharedPtr& cloud);
    void AddLivox(const livox_ros_driver2::msg::CustomMsg::SharedPtr& cloud);

    bool RunOnce(NavState* state);

    CloudPtr LatestDeskewedCloud() const;
    NavState LatestState() const;

private:
    bool SyncMeasurements();
    bool DeskewCurrentScan();
    bool MatchAgainstFixedMap(NavState* state);
    bool TrackingQualityGood() const;

```

```

    // fixed map ivox / kd-tree lives here
    // current scan must not be inserted into fixed map
};

} // namespace hikari::loclite

```

关键实现原则：

```

bool FastLioFixedMap::RunOnce(NavState* state) {
    if (!SyncMeasurements()) return false;
    if (!DeskewCurrentScan()) return false;
    if (!MatchAgainstFixedMap(state)) return false;

    // Lite fixed-map mode:
    // Do not call MapIncremental().
    // Do not add current scan points into fixed map.
    return true;
}

```

13. 固定地图加载片段

```

bool FastLioFixedMap::LoadFixedMap(const std::string& pcd_path, double voxel_leaf) {
    CloudPtr raw(new PointCloudType);
    if (pcl::io::loadPCDFile<PointType>(pcd_path, *raw) != 0) {
        return false;
    }

    CloudPtr filtered(new PointCloudType);
    pcl::VoxelGrid<PointType> voxel;
    voxel.setLeafSize(voxel_leaf, voxel_leaf, voxel_leaf);
    voxel.setInputCloud(raw);
    voxel.filter(*filtered);

    // Build fixed ivox / kdtree here.
    // ivox_->Clear();
    // ivox_->AddPoints(filtered->points);
    fixed_map_cloud_ = filtered;
    return true;
}

bool FastLioFixedMap::RebuildLocalMapAround(const SE3& T_map_lidar) {
    if (!fixed_map_cloud_) return false;

    const Eigen::Vector3d center = T_map_lidar.translation();
    CloudPtr local(new PointCloudType);
    local->reserve(fixed_map_cloud_->size());

    const double r2 = crop_radius_m_ * crop_radius_m_;
    for (const auto& p : fixed_map_cloud_->points) {
        const Eigen::Vector3d q(p.x, p.y, p.z);
        if ((q - center).squaredNorm() <= r2) {
            local->push_back(p);
        }
    }
}

// fixed_local_ivox_->Clear();

```

```

// fixed_local_ivox_ -> AddPoints(local->points);
fixed_local_map_cloud_ = local;
return true;
}

```

14. NDT Corrector 片段

```

#pragma once

#include <memory>
#include <string>

#include "hikari_loclite/common/nav_state.hpp"
#include "hikari_loclite/common/point_types.hpp"

namespace hikari::loclite {

struct NdtResult {
    bool valid = false;
    SE3 pose;
    double confidence = 0.0;
    double inlier_ratio = 0.0;
    double delta_trans_m = 0.0;
    double delta_rot_deg = 0.0;
};

class NdtCorrector {
public:
    using Ptr = std::shared_ptr<NdtCorrector>;

    bool Init(const std::string& yaml_path);
    bool SetMap(const CloudPtr& map);

    NdtResult Align(const CloudPtr& scan, const SE3& guess);
    NdtResult Validate(const SE3& candidate_pose, const CloudPtr& scan);

private:
    int threads_ = 1;
    double min_confidence_ = 1.0;
    double max_delta_trans_m_ = 1.0;
    double max_delta_rot_deg_ = 10.0;
};

} // namespace hikari::loclite

```

校验逻辑:

```

NdtResult NdtCorrector::Validate(const SE3& candidate_pose, const CloudPtr& scan) {
    auto r = Align(scan, candidate_pose);
    if (!r.valid) return r;

    const SE3 delta = candidate_pose.inverse() * r.pose;
    r.delta_trans_m = delta.translation().norm();
    r.delta_rot_deg = delta.so3().log().norm() * 180.0 / M_PI;

    if (r.confidence < min_confidence_) {

```

```

        r.valid = false;
    }
    if (r.delta_trans_m > max_delta_trans_m || r.delta_rot_deg > max_delta_rot_deg)
    {
        r.valid = false;
    }
    return r;
}

```

15. RelocManager 片段

```

#pragma once

#include <atomic>
#include <memory>
#include <string>

#include "hikari_loclite/common/nav_state.hpp"
#include "hikari_loclite/common/point_types.hpp"

namespace hikari::loclite {

struct RelocCandidate {
    bool valid = false;
    SE3 pose;
    double score = 0.0;
    std::string source;
};

class RelocManager {
public:
    using Ptr = std::shared_ptr<RelocManager>;

    bool Init(const std::string& yaml_path);

    void Arm(const char* reason) {
        armed_.store(true, std::memory_order_release);
        reason_ = reason ? reason : "";
    }

    void Disarm(const char* reason) {
        armed_.store(false, std::memory_order_release);
        reason_ = reason ? reason : "";
        ClearPendingJobs();
    }

    bool Armed() const {
        return armed_.load(std::memory_order_acquire);
    }

    bool AutoOnInit() const { return auto_on_init_; }

    RelocCandidate TryRelocalize(const CloudPtr& scan);
    RelocCandidate FromExternalPose(const SE3& pose);

private:
    void ClearPendingJobs();

```

```

std::atomic<bool> armed_{false};
bool auto_on_init_ = true;
bool auto_on_lost_ = true;
bool sc_enabled_ = true;
bool kiss_enabled_ = false;
std::string reason_;
};

} // namespace hikari::loclite

```

关键约束:

```

RelocCandidate RelocManager::TryRelocalize(const CloudPtr& scan) {
    if (!Armed()) return {};
    if (!scan || scan->empty()) return {};

    // Lite mode: run one bounded SC query, not a permanently active worker.
    // Top-K defaults to 1.
    // Candidate must be validated by NDT before ResetToMapPose().
    return RunScanContextOnce(scan);
}

```

16. Pose smoother 片段

```

#pragma once

#include "hikari_loclite/common/nav_state.hpp"

namespace hikari::loclite {

class LitePoseSmoother {
public:
    SE3 UpdateFastLioPose(const SE3& pose) {
        if (!initialized_) {
            last_pose_ = pose;
            initialized_ = true;
            return pose;
        }

        const SE3 delta = last_pose_.inverse() * pose;
        const double dt = delta.translation().norm();
        const double dr = delta.so3().log().norm() * 180.0 / M_PI;
        if (dt > max_output_jump_trans_m_ || dr > max_output_jump_rot_deg_) {
            return last_pose_;
        }

        last_pose_ = pose;
        return last_pose_;
    }

    SE3 ApplyNdtCorrection(const SE3& fast_pose, const SE3& ndt_pose, double gain) {
        const SE3 delta = fast_pose.inverse() * ndt_pose;
        const double dt = delta.translation().norm();
        const double dr = delta.so3().log().norm() * 180.0 / M_PI;
        if (dt > max_correction_trans_m_ || dr > max_correction_rot_deg_) {
            return fast_pose;
        }
    }
};

```



```

    }

    Eigen::Vector3d t = fast_pose.translation() * (1.0 - gain) +
ndt_pose.translation() * gain;
    SO3 r = fast_pose.so3() * SO3::exp(delta.so3().log() * gain);
    return SE3(r, t);
}

private:
    bool initialized_ = false;
    SE3 last_pose_;
    double max_output_jump_trans_m_ = 0.5;
    double max_output_jump_rot_deg_ = 15.0;
    double max_correction_trans_m_ = 0.3;
    double max_correction_rot_deg_ = 5.0;
};

} // namespace hikari::loclite

```

17. 构建命令

从工作空间根目录：

```

cd /home/aecriclin/3d_slam_ws
colcon build --packages-select hikari_loclite --cmake-args -
DCMAKE_BUILD_TYPE=Release

```

运行：

```

source /home/aecriclin/3d_slam_ws/install/setup.bash
ros2 run hikari_loclite run_loclite_online --config <yaml> --map_path <map_path>

```

如果使用 CPU 亲和性 / 实时调度：

```

sudo setcap cap_sys_nice+ep
/home/aecriclin/3d_slam_ws/install/hikari_loclite/lib/hikari_loclite/run_loclite_online

```

18. 分阶段实施建议

Phase 1: 独立包骨架

目标：

- 建 `hikari_loclite` ROS2 包。
- 能订阅 IMU/Lidar。
- 能发布空状态或模拟 pose。
- 不依赖 `lightning_libs`。

验收：

```
colcon build --packages-select hikari_loclite 通过
ros2 run 能启动
topic 订阅和状态发布正常
```

Phase 2: 搬 Fast-LIO 最小前端

目标:

- 抽取 Fast-LIO ESKF、预处理、iVox 匹配的最小代码。
- 跑 incremental local map，先证明前端可用。

验收:

```
输入 Livox + IMU 能输出连续 odom
CPU 和 full lightning 对比明显更低
```

Phase 3: fixed-map

目标:

- 加载 `global.pcd`。
- 构建 fixed iVox。
- 禁止当前 scan 加入地图。
- 已知初值下稳定 tracking。

验收:

```
fixed map 点数不增长
输出 pose 在 map 坐标系
重启后复现一致
```

Phase 4: /initialpose + NDT validation

目标:

- 接收 `/initialpose`。
- NDT 验证初值。
- 通过后 `ResetToMapPose()`。

验收:

```
正确初值进入 Good
错误初值被拒绝
不会污染 Fast-LIO 状态
```

Phase 5: SC init / LOST

目标:

- 离线加载 SC 数据库。
- init / LOST 时运行一次 bounded SC query。

- 候选经 NDT 验证。
- Good 后 `RelocManager::Disarm()`。

验收：

```
Good 后无 SC 线程占 CPU
Lost 后能重新 Arm
重定位成功后再次 Disarm
```

Phase 6: 产品化

目标：

- systemd service。
- setcap。
- CPU affinity。
- 日志限频。
- 参数模板。
- watchdog 状态上报。

19. 与现有 full 框架的边界

`hikari_loclite` 不应直接 include：

```
core/system/loc_system.h
core/localization/localization.h
core/localization/lidar_loc/lidar_loc.h
core/localization/pose_graph/pgo.h
ui/pangolin_window.h
```

可参考但应重写/抽小：

```
LocSystem 的 ROS topic 接入方式
LaserMapping 的 Fast-LIO 前端
LidarLoc 的 NDT_OMP 调用方式
ScanContextManager 的数据库格式
```

这样可以避免新包变成 full 包的另一层封装。

20. 最终推荐

直接做独立 `hikari_loclite` 是合理选择。建议路线：

```
不在 full lightning 上继续打补丁
不链接 lightning.libs
新包只抽最小 Fast-LIO fixed-map 能力
NDT 做低频验证/校正
SC 做 init/LOST 一次性重定位
Good 后无重定位线程、无 PGO、无动态地图
```

这条路线的工程收益：

- 编译更轻。
- 部署更清楚。
- CPU 预算更可控。
- Full 系统仍可保留为建图、调试、回归验证工具。

21.与 Lighting的接口验证

我们约定 hikari_loclite 的启动参数和上抛ROS2话题和TF行为树与 lightning 定位模式 一致，即可做到直接替换现有 lightning 定位模式做验证。

即有：

run_loc_offline — 离线定位评估

注入 rosbag 数据与先验地图，用于评估定位精度与算法稳定性。

```
ros2 run hikari_loclite run_loclite_offline --config <yaml> --input_bag <bag>
```

定位模式 (run_loc_online)

```
ros2 run hikari_loclite run_loclite_online --config <yaml> --map_path <map_path>
```

话题名	消息类型	说明
hikari_loc/path	nav_msgs/Path	定位输出轨迹路径，Frame:map。最大保存 5000 个位姿。
hikari_loc/loc_state	std_msgs/Int32	定位状态枚举整数：0=IDLE, 1=INITIALIZING, 2=GOOD, 3=DEGRADED, 4=LOST, 5=WAIT_FOR_INITIALPOSE
hikari_loc/ndt_status	std_msgs/Int32	当前 NDT 级联层级：0=默认 1.0m, 1=级联模式
hikari_loc/loc_status	visualization_msgs/Marker	RViz 文本标记，显示定位状态文字（GOOD/DEGRADED/INIT/LOST/WAIT FOR /initialpose）与置信度值，颜色编码：绿/黄/蓝/红/紫
hikari_loc/sc/accum_cloud	sensor_msgs/PointCloud2	ScanContext 累积点云（重力对齐后），调试用。仅在有订阅者时发布。
hikari_loc/sc/candidates	visualization_msgs/MarkerArray	ScanContext 候选位姿箭头：绿=胜出，白=拒绝，红=重力检查失败。仅在有订阅者时发布。
hikari_loc/sc/init_guess	geometry_msgs/PoseStamped	ScanContext 初始猜测位姿（胜出的 SC 候选）。仅在有订阅者时发布。

话题名	消息类型	说明
<code>hikari_loc/sc/gravity_check</code>	<code>std_msgs/Float32MultiArray</code>	SC 重力一致性检查结果： [roll_err_deg, pitch_err_deg, passed]， passed 为 1.0 或 0.0。仅在有订阅者时发布。

TF 变换广播

父 Frame	子 Frame	节点	说明
<code>lidar_frame</code>	<code>level_frame</code>	<code>run_loclite_online</code>	重力对齐水平坐标系，纯旋转变换。Frame ID 可由 <code>system.lidar_frame_id</code> 和 <code>system.level_frame_id</code> 配置。
<code>map</code>	<code>base_link</code>	<code>run_loclite_online</code>	定位输出位姿（补偿了 <code>base_link</code> → <code>lidar</code> 外参后的真实 <code>base_link</code> 位姿）。

默认使用lightning的地图格式 `map_path` 传入地图文件夹：

```
data/new_map/
├─ global.pcd          # 全局点云地图
├─ map.pgm             # 2D 栅格地图（pgm 格式）
├─ map.yaml            # 2D 地图元数据（分辨率、原点等）
├─ poses.txt           # 关键帧位姿序列
├─ sc_database.bin      # Scan Context 数据库（定位时使用）
├─ keyframes/          # 关键帧点云目录
└─ <tiles>/            # 分块瓦片点云
```

定位状态机

系统定义了以下定位状态（通过 `hikari_loc/loc_state` 话题发布）：

状态值	状态名	说明
0	IDLE	空闲，等待初始化
1	INITIALIZING	正在初始化（NDT 匹配 / 稳定门控未放行 / SC 重定位中）
2	GOOD	定位正常
3	DEGRADED	定位退化（跟踪质量下降但仍在运行）
4	LOST	定位丢失（系统内部仍在尝试 SC / FP 自救，未超 <code>lost_timeout_sec</code> 时长）
5	WAIT_FOR_INITIALPOSE	LOST 超时或 FP 兜底超时后冻结后端， 仅接受外部 <code>/initialpose</code> 。1Hz 节流持续上报。

状态升级规则: LOST 持续 > `lost_timeout_sec`、或 SC 耗尽后 FP 兜底超过 `fp_fallback_timeout_sec` → 进入 WAIT_FOR_INITIALPOSE。RViz 紫色 marker `WAIT FOR /initialpose` 即此状态。

初始位姿注入 (`/initialpose`)

`/initialpose` 在系统中被视作 "指令" (command)，不是"建议"。语义保证如下：

- 1. **限次重试 (sticky retry)** — FC 单帧拒绝不会立刻丢弃 pose。lidar_loc.init_max_retries 次内会用后续帧的新 scan 反复重试同一 pose，直到通过或耗尽。
- 2. **不会跌落到 FP 暴力搜索** — 即使所有重试失败，系统也不会去地图里找另一个 NDT 分高的关键帧覆盖用户意图。FP 暴力搜索仅在用户从未给过 /initialpose 的冷启动场景下运行。
- 3. **SC blackout 窗口 (system.external_pose_blackout_sec)** — 注入后此秒数内禁止 SC worker 抢注：ScanContextReloc 入口阻断 + SC inflight job 即便算完也丢弃，让用户的 pose 拥有独占试错窗口。
- 4. **状态机原子复位** — WAIT / LOST / SC 失败计数 / FP 超时计时器全部清零；StabilityGate re-arm。

注入方式:

```
# RViz "2D Pose Estimate" 工具最常用; 或手动:
ros2 topic pub --once /initialpose geometry_msgs/PoseWithCovarianceStamped \
  "{header: {frame_id: 'map'}, pose: {pose: {position: {x: 0.0, y: 0.0, z: 0.0},
orientation: {w: 1.0}}}}"
```

注入后日志关键字 (按出现顺序):

日志	含义
[ExtPose] /initialpose 已接入 (t=...): ... blackout 窗口至 t=...	SetExternalPose 落地, blackout 起止时间
Set initial pose is: [...]	LidarLoc 接收到指令
[ExtPose] 处于 blackout 窗口, 本帧跳过 SC 全流程	每 20 帧节流, 确认 force_fc 持续生效
[FC-Init] FC 拒绝本帧, 保留 init pose 用下一帧重试 (k/N)	单帧失败, 正在重试
init with external pose: [...] (after k retries)	第 k 次重试成功
[FC-Init] /initialpose 重试 N 次仍失败, 放弃, 通知上层 rearm	N 次仍失败
[SC-Reloc/worker] 处于 /initialpose blackout 窗口 ..., 丢弃本次 SC 注入	SC inflight job 被 blackout 拦掉

手动 SC 重定位

当定位丢失且不使用 RViz 时, 可手动触发 ScanContext 重定位:

```
ros2 service call /hikari_loc/sc_reloc std_srvs/srv/Trigger "{}"
```

注意: 该服务路径**绕过 blackout 窗口** (用户显式触发即视为授权抢注)。

注意 在release模式下 offline节点不编译 在 release 模式下也不带编译Pangolin UI

要求 在非 release模式下编译 Pangolin UI 且 UI行为与 Lighting 一致, 用于测试调试。